ENGENHARIA DE SOFTWARE - 2026.2

SISTEMA DE GERENCIAMENTO DE EVENTOS CIENTÍFICOS USANDO MICROSSERVIÇOS E COMPONENTES DE SOFTWARE

Arthur Carvalho e Ana Leticia Nobre

Repositório do projeto:

https://github.com/arthuurca/projeto_Engenharia_de_software

2026.2

1. APRESENTAÇÃO E ENUNCIADO

Desenvolvemos um sistema web para cadastrar eventos científicos, registrar participantes, controlar inscrições e recomendar eventos por área de interesse. Nosso objetivo foi construir uma versão pequena, com um fluxo completo de uso, demonstrando microsserviços, API Gateway e componentes reutilizáveis.

Os organizadores precisam divulgar os dados dos eventos e acompanhar suas vagas. Os participantes precisam consultar os eventos, realizar inscrições e encontrar atividades relacionadas aos seus interesses. Reunimos essas operações em uma interface simples.

O organizador pode cadastrar eventos e consultar a programação. O participante pode cadastrar seus dados, selecionar seu nome para a demonstração, consultar eventos, inscrever-se e acompanhar suas inscrições e recomendações. Nesta versão acadêmica local, não implementamos autenticação ou controle de permissões. Utilizamos dados fictícios.

O participante possui nome, e-mail e uma área de interesse. O evento possui título, descrição, data e horário de início, local, área e capacidade. Definimos as áreas Computação, Engenharias, Saúde e Ciências Humanas. Não incluímos edição, exclusão, cancelamento de inscrições, pagamentos, certificados, submissão de artigos, envio de e-mails ou aplicativo móvel. As recomendações usam regras simples.

2. METODOLOGIA E ORGANIZAÇÃO DO TRABALHO

Não foi definida nem registrada uma duração fixa para cada sprint. As três etapas foram delimitadas pelos objetivos e entregas, como uma adaptação acadêmica, e não como ciclos formais de duração fixa do Scrum.

Organizamos o trabalho em três sprints, com um processo ágil simplificado inspirado em Scrum. Usamos o Product Backlog para definir o que construir e os Sprint Backlogs para dividir a execução. O acompanhamento considera os estados A fazer, Em andamento e Concluído. Não adotamos todos os papéis e eventos formais do Scrum; a organização foi adaptada a um trabalho em dupla.

Na divisão inicial, Arthur ficou como responsável previsto pelos microsserviços, pelo Gateway e pelas instruções de execução, e Ana pela interface, pelos componentes e pela organização do relatório. Requisitos, arquitetura, integração, testes e revisão foram definidos como atividades compartilhadas. A divisão serviu como referência para organizar as tarefas e acompanhar os resultados.

Na Sprint 1, foram implementados Gateway, Usuários, Eventos e as telas de cadastro e consulta. Na Sprint 2, foram adicionados Inscrições, Recomendações e o cálculo real de vagas. Nesta revisão da Sprint 3, conferimos os testes, a documentação e os materiais de entrega. O endereço do repositório foi incluído na capa para acompanhar a publicação do código.

3. REQUISITOS E PRODUCT BACKLOG

No projeto, aplicamos reuso por meio dos microsserviços e dos componentes de formulário, cartão de evento e comunicação com a API.

Requisitos funcionais

ID	Requisito	Serviço responsável
RF01	Cadastrar participante com nome, e-mail e uma área de interesse.	Usuários

ID	Requisito	Serviço responsável
RF02	Listar participantes para selecionar quem utilizará a área do participante.	Usuários
RF03	Cadastrar evento com título, descrição, data e horário de início, local, área e limite de vagas.	Eventos
RF04	Listar eventos com título, data, local, área e vagas disponíveis.	Eventos, com ocupação fornecida por Inscrições
RF05	Registrar uma inscrição do participante selecionado em um evento.	Inscrições
RF06	Listar as inscrições do participante selecionado, apresentando os dados dos eventos.	Inscrições, com dados de Eventos
RF07	Recomendar eventos de acordo com a área de interesse do participante.	Recomendações, consultando Usuários, Eventos e Inscrições

Os serviços responsáveis poderão consultar os demais por suas APIs. Os contratos estão documentados na arquitetura. A navegação entre organizador e participante não constitui autenticação nem controle de acesso.

Regras de negócio

ID	Regra
RN01	Todos os campos dos cadastros são obrigatórios. Textos compostos apenas por espaços são inválidos; o e-mail deve ter formato válido.
RN02	O e-mail do participante deve ser único, desconsiderando maiúsculas, minúsculas e espaços nas extremidades.
RN03	A data e o horário de início do evento devem ser futuros no momento do cadastro. O limite de vagas deve ser um número inteiro maior que zero.
RN04	Uma inscrição exige participante e evento existentes.
RN05	Só pode existir uma inscrição para cada combinação de participante e evento.
RN06	Inscrições são permitidas somente antes do início do evento e enquanto houver vagas. Solicitações simultâneas não podem ultrapassar a capacidade.
RN07	Recomendações incluem apenas eventos futuros, com vagas, da mesma área de interesse e sem inscrição do participante selecionado.
RN08	Participantes e eventos devem ter uma área da lista fixa: Computação, Engenharias, Saúde ou Ciências Humanas.
RN09	Vagas disponíveis correspondem ao limite do evento menos a quantidade de inscrições confirmadas. Uma tentativa recusada não consome vaga.

Datas e horários serão apresentados no fuso de Brasília (America/Sao_Paulo). O sistema deve comparar instantes de maneira consistente; o armazenamento em UTC está definido na arquitetura.

Product Backlog

Todos os seis itens são obrigatórios para a versão mínima definida pela dupla. A ordem abaixo considera as dependências entre funcionalidades. Todos os itens foram implementados e verificados nas sprints 1 e 2.

Ordem / ID	História de usuário	Requisitos	Dependências
1 - PB01	Como participante, quero cadastrar meus dados e selecionar meu cadastro para utilizar o sistema.	RF01, RF02; RN01, RN02, RN08	Nenhuma
2 - PB02	Como organizador, quero cadastrar um evento para divulgá-lo.	RF03; RN01, RN03, RN08	Nenhuma
3 - PB03	Como participante, quero consultar os eventos e suas vagas para escolher um evento.	RF04; RN09	PB02; a ocupação é integrada com PB04
4 - PB04	Como participante, quero me inscrever em um evento para reservar uma vaga.	RF05; RN04, RN05, RN06, RN09	PB01, PB02, PB03
5 - PB05	Como participante, quero consultar minhas inscrições para acompanhar meus eventos.	RF06	PB04
6 - PB06	Como participante, quero receber sugestões relacionadas ao meu interesse para encontrar eventos relevantes.	RF07; RN07, RN08	PB01, PB02, PB04

PB01 - Cadastro e seleção do participante

- Com nome, e-mail válido e área permitida, o cadastro é salvo e aparece na lista de participantes.
- Ao selecionar um participante, a interface identifica o cadastro usado nas inscrições e recomendações.
- Campos vazios, e-mail inválido e área não permitida são recusados com mensagem compreensível.
- Um e-mail já cadastrado é recusado, inclusive quando escrito com diferenças de maiúsculas ou espaços nas extremidades.
- Sem participantes cadastrados, a interface informa essa situação e permite acessar o cadastro.

PB02 - Cadastro do evento

- Com todos os campos válidos, o evento é salvo e aparece na listagem.
- Campos vazios e área não permitida são recusados.
- Data passada ou igual ao instante atual é recusada.
- Capacidade zero, negativa ou fracionária é recusada.

PB03 - Consulta dos eventos

- Cada evento apresenta título, data e horário, local, área e vagas disponíveis.
- Um evento sem inscrições apresenta todas as suas vagas disponíveis.
- Após uma inscrição confirmada, a consulta atualizada apresenta uma vaga a menos.
- Eventos iniciados ou lotados podem continuar na listagem, mas não permitem nova inscrição.
- Sem eventos cadastrados, a interface apresenta uma mensagem de lista vazia.

PB04 - Inscrição

- Com participante e evento existentes, data futura e vaga disponível, a inscrição é confirmada e reserva exatamente uma vaga.
- Uma nova inscrição da mesma pessoa no mesmo evento é recusada sem alterar a ocupação.
- Evento lotado ou iniciado recusa a inscrição sem criar registro.
- Participante ou evento inexistente recusa a operação sem criar registro.
- Para um evento com uma vaga restante, duas solicitações simultâneas de participantes diferentes geram apenas uma confirmação.
- Sem participante selecionado, a interface orienta a seleção antes de enviar a inscrição.

PB05 - Consulta das inscrições

- A consulta apresenta somente inscrições do participante selecionado, com título, data e local dos respectivos eventos.
- Ao trocar o participante selecionado, a listagem corresponde ao novo participante.
- Sem inscrições, a interface informa que o participante ainda não está inscrito em eventos.

PB06 - Recomendação

- Para um participante interessado em Computação, um evento futuro dessa área, com vaga e sem inscrição prévia, aparece como sugestão.
- Eventos de outra área, iniciados, lotados ou com inscrição do participante não aparecem nas sugestões.
- Após uma inscrição, a consulta atualizada de recomendações deixa de sugerir esse evento.
- Sem eventos elegíveis, a interface informa que não há recomendações disponíveis.
- As sugestões são calculadas por regras simples, sem serviço externo.

Requisitos técnicos e de qualidade

Os requisitos abaixo orientaram as verificações do sistema.

ID	Requisito	Como verificar na implementação
RQ01	A interface acessa as APIs de negócio exclusivamente pela API Gateway.	Inspecionar as chamadas da interface e demonstrar o encaminhamento aos serviços.
RQ02	Usuários, Eventos, Inscrições e Recomendações executam separadamente e oferecem verificação de estado.	Iniciar os quatro processos e consultar suas rotas de estado; capturar a listagem dos serviços em execução.
RQ03	Participantes, eventos e inscrições possuem persistência.	Cadastrar dados, reiniciar os serviços e consultar os mesmos registros.
RQ04	A interface apresenta mensagens de validação e indisponibilidade sem informar sucesso quando a operação falhar.	Enviar dados inválidos e interromper um serviço necessário para uma consulta ou operação.
RQ05	A interface reutiliza cartão de evento, campo de formulário e cliente de API.	Identificar seus usos no código e documentá-los no relatório.
RQ06	A tela principal identifica os microsserviços utilizados.	Verificar a identificação na interface e registrar uma captura de tela.
RQ07	Gateway, microsserviços, componentes e regra de recomendação têm suas responsabilidades e uso documentados.	Revisar documentação de código, endpoints e instruções de execução.

4. SPRINT BACKLOGS E RESULTADOS

O reuso acompanhou as etapas de desenvolvimento. Na primeira sprint, criamos os campos dos cadastros e o cliente de API compartilhado. Na segunda, reutilizamos o cartão de evento nas inscrições e recomendações. Na revisão final, conferimos a integração e documentamos as responsabilidades desses componentes.

Mantivemos abaixo os objetivos e as tarefas que orientaram as sprints. As entregas esperadas descrevem o planejamento; a situação efetiva está registrada logo depois de cada etapa no resumo de resultados.

Sprint 1 - Base e cadastros

Objetivo: permitir cadastrar participantes e eventos e consultar os cadastros pela interface. Itens: PB01, PB02 e PB03. A atualização das vagas após inscrições será concluída na Sprint 2.

ID	Tarefa	Responsável previsto
S1-01	Escolher tecnologias e persistência; desenhar arquitetura e definir contratos das APIs.	Ambos
S1-02	Organizar o repositório e implementar a API Gateway e os serviços de usuários e eventos.	Arthur
S1-03	Criar telas de cadastro, seleção de participante e listagem de eventos, com componentes reutilizáveis.	Ana
S1-04	Integrar as telas ao Gateway e verificar cadastros válidos, inválidos, e-mail duplicado e persistência após reinício.	Ambos
S1-05	Documentar a arquitetura inicial, a execução e os resultados reais da sprint.	Ambos

Entrega esperada: cadastros e listagens funcionando, com dados persistidos e comunicação pela API Gateway.

Sprint 2 - Inscrições e recomendações

Objetivo: completar o fluxo do participante. Itens: PB04, PB05, PB06 e integração final de PB03.

ID	Tarefa	Responsável previsto
S2-01	Implementar o serviço de inscrições com validação de existência, duplicidade, início do evento e capacidade.	Arthur
S2-02	Implementar o serviço de recomendações com filtros de área, data, vagas e inscrições existentes.	Arthur
S2-03	Criar e integrar as ações de inscrição e as telas de minhas inscrições e recomendações.	Ana
S2-04	Integrar vagas disponíveis à listagem de eventos e identificar os serviços na tela principal.	Ambos
S2-05	Verificar inscrições, isolamento da consulta por participante, recomendações e disputa simultânea pela última vaga.	Ambos
S2-06	Atualizar a documentação e registrar resultados e pendências.	Ambos

Entrega esperada: quatro microsserviços integrados ao Gateway, com inscrição, consulta e recomendação funcionando pela interface.

Sprint 3 - Validação e entrega

Objetivo: revisar o funcionamento e preparar a entrega solicitada pelo professor.

ID	Tarefa	Responsável previsto
S3-01	Executar todos os critérios de aceitação e verificar mensagens de erro e indisponibilidade.	Ambos
S3-02	Corrigir falhas encontradas e revisar persistência e execução independente dos serviços.	Ambos
S3-03	Revisar a documentação do Gateway, dos serviços, dos componentes e da regra de recomendação.	Ambos

ID	Tarefa	Responsável previsto
S3-04	Registrar telas do fluxo completo e a listagem dos microsserviços em execução.	Ana, com revisão de Arthur
S3-05	Finalizar as instruções e publicar o código funcional no GitHub; validar a execução seguindo o README.	Arthur, com revisão de Ana
S3-06	Completar o relatório, incluir o link do GitHub na capa e ensaiar a demonstração.	Ambos

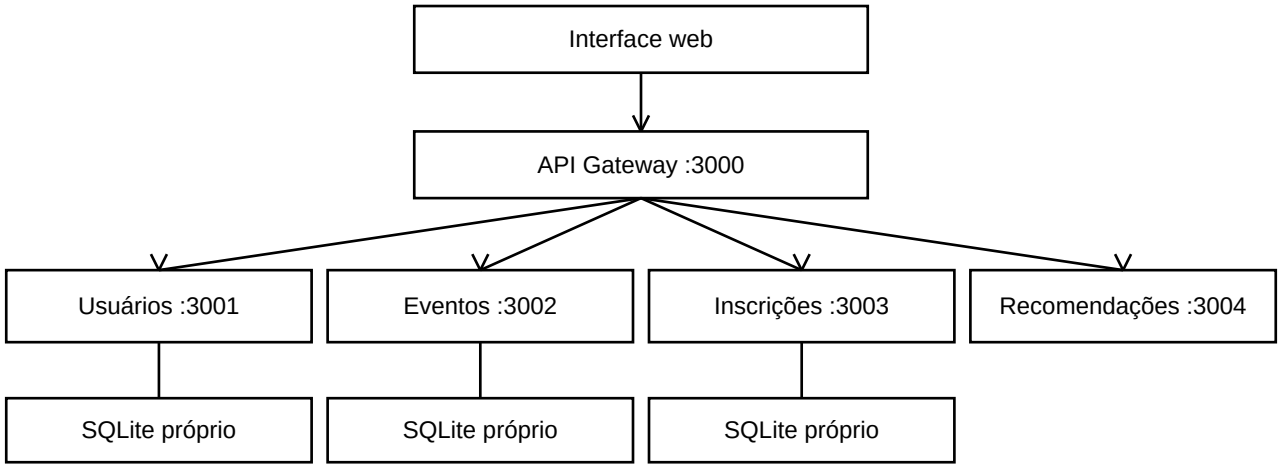
Entrega esperada: sistema verificado, código no GitHub, instruções de execução e relatório completo com evidências.

Resultados da Sprint 1: S1-01 a S1-05 implementados e documentados. PB01 e PB02 concluídos tecnicamente. PB03 foi iniciado com consulta de eventos e completado na Sprint 2.

Resultados da Sprint 2: S2-01 a S2-06 implementados e documentados. PB03 a PB06 concluídos tecnicamente, incluindo vagas, inscrição, consultas pessoais e sugestões.

Resultados da revisão da Sprint 3:
- S3-01: testes de integração aprovados, com validações e indisponibilidade.
- S3-02: regras principais, persistência e execução independente verificadas.
- S3-03: responsabilidades, endpoints, componentes e recomendação documentados.
- S3-04: capturas reais da interface e dos quatro serviços online organizadas.
- S3-05: instruções e pacote de código preparados; código publicado no repositório indicado na capa.
- S3-06: relatório e roteiro preparados; link do GitHub incluído; roteiro de demonstração concluído.

5. ARQUITETURA DO SISTEMA



Chamadas internas por HTTP: Inscrições → Usuários e Eventos; Eventos → Inscrições (ocupação); Recomendações → Usuários, Eventos e Inscrições. Cada seta do diagrama parte do chamador. As respostas retornam pela mesma conexão.

Cada serviço executa em um processo separado. A interface faz as chamadas de negócio somente pelo Gateway. Os serviços consultam as APIs dos demais quando precisam de informações; cada banco é acessado somente pelo serviço proprietário.

Inscrições consulta Usuários e a rota básica de Eventos antes de confirmar. A listagem de Eventos consulta a ocupação em Inscrições. Recomendações consulta interesses, eventos com vagas e IDs das inscrições. As

rotas internas de ocupação não fazem novas consultas a Eventos, evitando chamadas circulares.

Os processos escutam localmente em 127.0.0.1, com portas configuráveis. Não utilizamos Docker ou um servidor separado de banco nesta versão.

6. TECNOLOGIAS UTILIZADAS

Utilizamos Python 3.12, FastAPI e Uvicorn no Gateway e nos quatro microsserviços. Usuários, Eventos e Inscrições possuem bancos SQLite separados, acessados pela biblioteca sqlite3. Gateway e Recomendações não possuem banco. A comunicação interna usa HTTPX; a interface usa HTML, CSS, JavaScript e fetch. As versões instaladas estão em requirements-python-lock.txt.

Processo	Linguagem do backend	Framework/servidor	Banco
Gateway	Python 3.12	FastAPI/Uvicorn	Nenhum
Usuários	Python 3.12	FastAPI/Uvicorn	SQLite: usuarios.sqlite
Eventos	Python 3.12	FastAPI/Uvicorn	SQLite: eventos.sqlite
Inscrições	Python 3.12	FastAPI/Uvicorn	SQLite: inscricoes.sqlite
Recomendações	Python 3.12	FastAPI/Uvicorn	Nenhum

7. REUSO, ESPECIFICAÇÃO E CÓDIGO DOCUMENTADO

O Gateway em python/gateway/app.py entrega a interface e encaminha as rotas /api/participantes, /api/eventos, /api/inscricoes e /api/recomendacoes. /api/status consulta o estado dos quatro serviços. Cada processo possui /health.

Os serviços estão em python/services/usuarios/app.py, eventos/app.py, inscricoes/app.py e recomendacoes/app.py. Usuários cadastra e consulta participantes; Eventos cadastra e consulta eventos e obtém ocupação em Inscrições; Inscrições valida referências, data, duplicidade e capacidade; Recomendações filtra por área, data futura, vagas e ausência de inscrição prévia.

Sistema de recomendação

O serviço executa os seguintes passos: 1) consulta a área de interesse do participante; 2) consulta os eventos e suas vagas; 3) obtém os eventos em que o participante já está inscrito; 4) seleciona eventos futuros da mesma área, com vagas e sem inscrição prévia; 5) ordena pelo início e pelo identificador. Se não houver resultados, retorna uma lista vazia. Uma falha de comunicação gera uma mensagem de erro.

python/common.py compartilha validações, respostas de erro, conexão SQLite e cliente HTTP. python/shared.py reúne áreas e configuração. python/start.py inicia os cinco processos. Os componentes frontend/components/field.js e event-card.js e o cliente frontend/api.js são reutilizados pelos formulários, programação, inscrições e sugestões.

A inscrição verifica referências antes de BEGIN IMMEDIATE, conta inscrições e insere dentro da mesma transação, sem chamada HTTP durante a transação. Erros causam ROLLBACK. A restrição UNIQUE também protege o par participante/evento. A capacidade não muda nesta versão sem edição. As rotas internas de ocupação e IDs evitam chamadas circulares.

O código completo está no repositório. Os códigos solicitados pelo professor estão no Anexo B deste relatório. Os formatos continuam sendo {dados: ...} e {erro: {codigo, mensagem}}. Dados inválidos retornam 400; inexistentes, 404; duplicidade ou lotação, 409; indisponibilidade, 503; tempo excedido, 504.

8. TELAS E FUNCIONAMENTO PASSO A PASSO

Tela 1 - Página principal e serviços: a página identifica a aplicação e apresenta o estado real de Usuários, Eventos, Inscrições e Recomendações consultado pelo Gateway. O painel está na Figura 1 e também está destacado na Figura 7 do Anexo A.

Tela 2 (Figura 2 do Anexo A) - Participante: informamos nome, e-mail e área de interesse. O cadastro salvo aparece na seleção. Ao trocar o participante selecionado, suas inscrições e sugestões são atualizadas. Essa seleção é apenas um recurso da demonstração local, sem login.

Tela 3 (Figura 3 do Anexo A) - Organizador: informamos título, descrição, data e horário futuros, local, área e capacidade. O servidor repete as validações da interface e recusa dados inválidos.

Tela 4 (Figura 4 do Anexo A) - Programação: os cartões mostram dados dos eventos e vagas disponíveis. O botão Inscrever-se envia o participante e o evento ao serviço de Inscrições por meio do Gateway. O serviço valida novamente a capacidade antes de confirmar.

Tela 5 (Figura 5 do Anexo A) - Minhas inscrições: após a confirmação, a inscrição aparece na consulta do participante selecionado. Uma nova tentativa para o mesmo evento é recusada; o botão também passa a indicar Inscrito.

Tela 6 (Figura 6 do Anexo A) - Recomendações: eventos futuros da mesma área, com vagas e sem inscrição prévia são sugeridos. Depois da inscrição, o evento escolhido sai dessa lista. Sem resultados, a interface informa que não há recomendações.

As Figuras 1 a 6 do Anexo A apresentam os serviços e o fluxo de utilização. A Figura 7 mostra a página inteira com o painel de microsserviços destacado. A Figura 6 mostra um evento recomendado antes da inscrição; a ausência de sugestões após a inscrição corresponde ao filtro de eventos já escolhidos.

9. TESTES E EXECUÇÃO

A suíte Python passou com dois métodos de teste, sendo o segundo composto por sete cenários. Verificamos cadastros, validações, duplicidade, referências, inscrições, consultas por participante, recomendações, concorrência pela última vaga, persistência após reiniciar os serviços e indisponibilidade de dependências. O registro detalhado está em docs/13-validacao-python.md.

No navegador Python, cadastramos Aluno Teste Python e Simpósio Python. A inscrição reduziu as vagas de duas para uma, apareceu em Minhas inscrições e saiu das recomendações. O fluxo está ilustrado nas Figuras 1 a 6 do Anexo A.

Para executar:
1. Instalar Python 3.12 com pip.
2. Executar py -3.12 -m venv .venv.
3. Executar .venv\Scripts\python.exe -m pip install -r requirements-python-lock.txt.
4. Executar .venv\Scripts\python.exe tests\python_migration.py -v.
5. Executar .venv\Scripts\python.exe -m python.start.
6. Abrir http://127.0.0.1:3000 e utilizar dados fictícios.

Uma instalação nova começa sem cadastros. Os bancos são criados em data/. As portas e DATA_DIR são configuráveis. A interface não possui autenticação; o participante selecionado identifica apenas o cadastro usado na demonstração local.

10. SITUAÇÃO DA ENTREGA

O código funcional, as instruções, os backlogs, a arquitetura, a documentação dos componentes e as evidências estão organizados no projeto. O vídeo exigido especificamente para jogos não se aplica ao nosso sistema de eventos.

O repositório de entrega é https://github.com/arthuurca/projeto_Engenharia_de_software, também indicado na capa. O código, os relatórios e as evidências acompanham a entrega. O acesso público ao repositório foi confirmado por consulta sem autenticação à API do GitHub.

11. REFERÊNCIAS

Material da disciplina. Orientações do Projeto 2026.2, fornecidas em imagem e guardadas em fontes/orientacoes-projeto-2026-2.png.
Material da disciplina. Elementos do Relatório do Projeto utilizando Microservices. PDF fornecido para o trabalho, disponível na pasta fontes.
FASTAPI. Documentação oficial. https://fastapi.tiangolo.com/. Consulta em 31 out. 2026.
PYTHON. sqlite3, documentação do Python 3.12. https://docs.python.org/3.12/library/sqlite3.html. Consulta em 31 out. 2026.
HTTPX. Documentação oficial. https://www.python-httpx.org/. Consulta em 31 out. 2026.

O tema e os integrantes foram registrados em fontes/tema-e-integrantes.md. As regras e os limites específicos são decisões do escopo do nosso projeto.

REFERÊNCIAS COMPLEMENTARES DE ENGENHARIA DE SOFTWARE

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2019. Referência complementar para requisitos, projeto e validação.
NEWMAN, Sam. Building Microservices: Designing Fine-Grained Systems. 2. ed. O’Reilly Media, 2021. Referência complementar sobre separação de serviços e comunicação.
SCHWABER, Ken; SUTHERLAND, Jeff. The Scrum Guide. Novembro de 2020. Disponível em: https://scrumguides.org/scrum-guide.html. Consulta em 31 out. 2026. A organização adotada aqui é simplificada e não corresponde à aplicação integral do Scrum.

ANEXO A - TELAS DO SISTEMA

Figura 1 - Painel com os quatro microsserviços online.

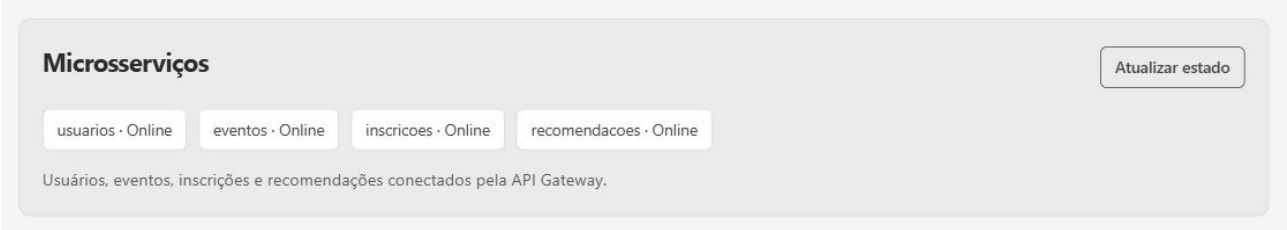


Figura 2 - Cadastro e seleção do participante.

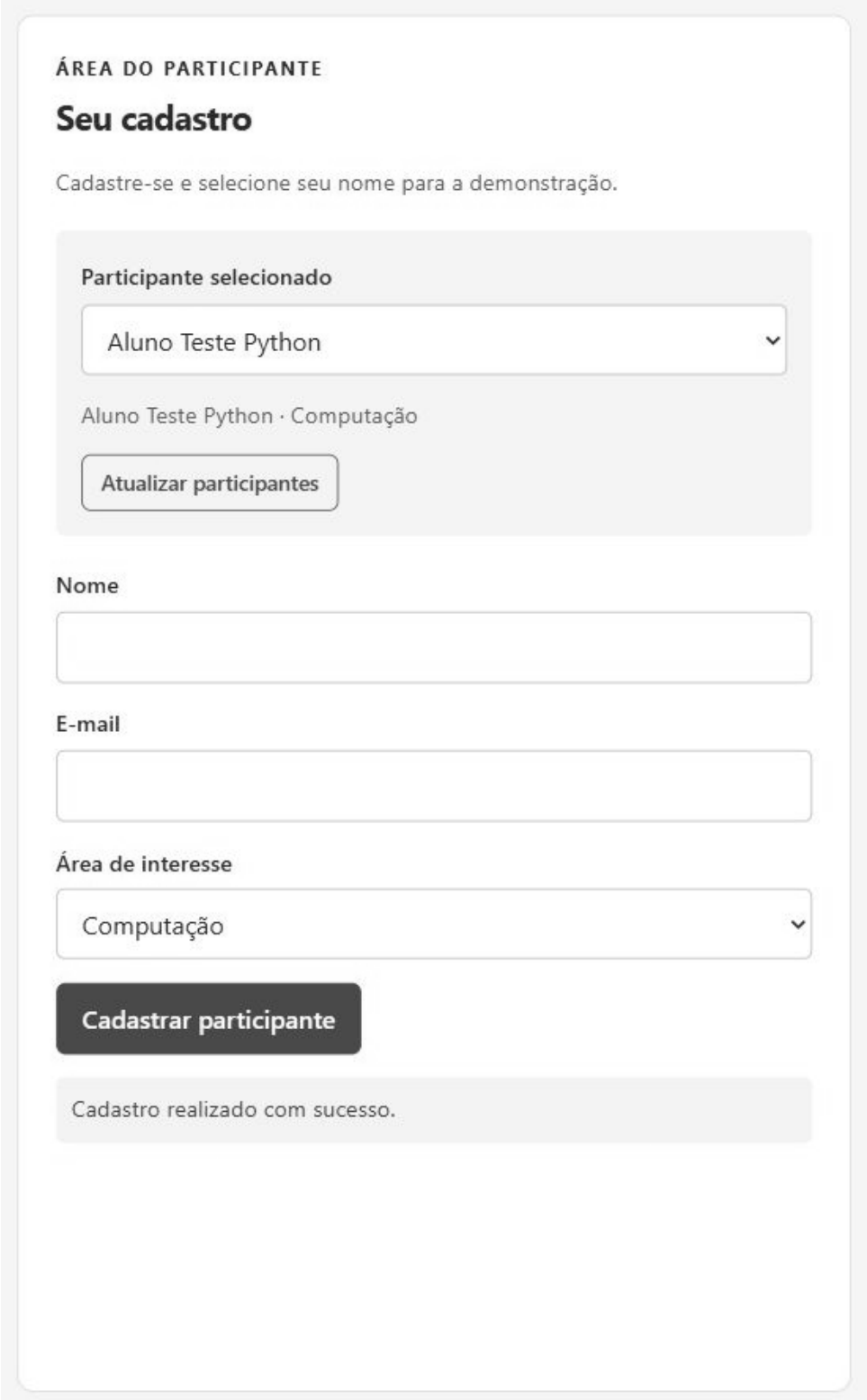


Figura 3 - Cadastro de evento.

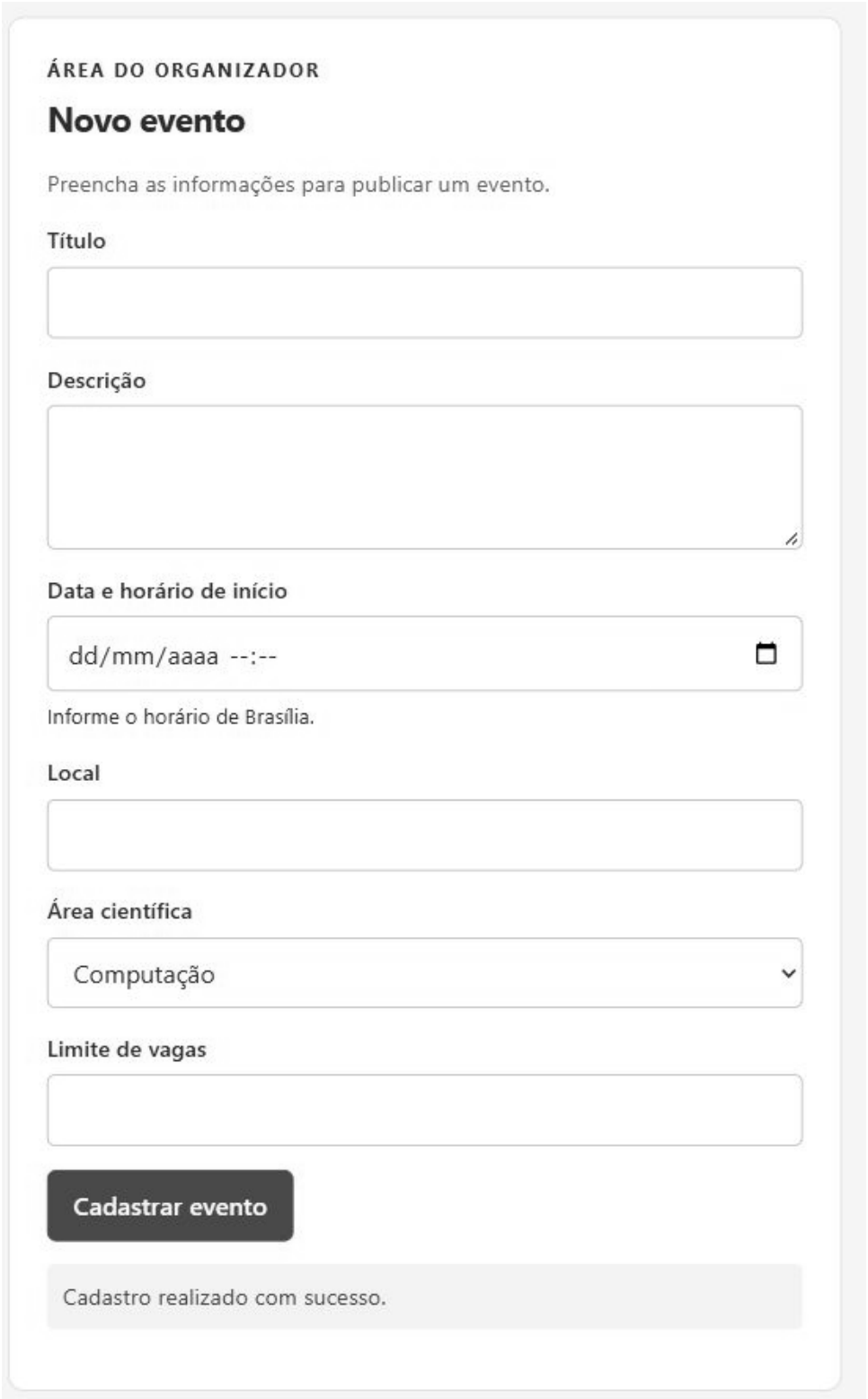


Figura 4 - Programação e confirmação da inscrição.

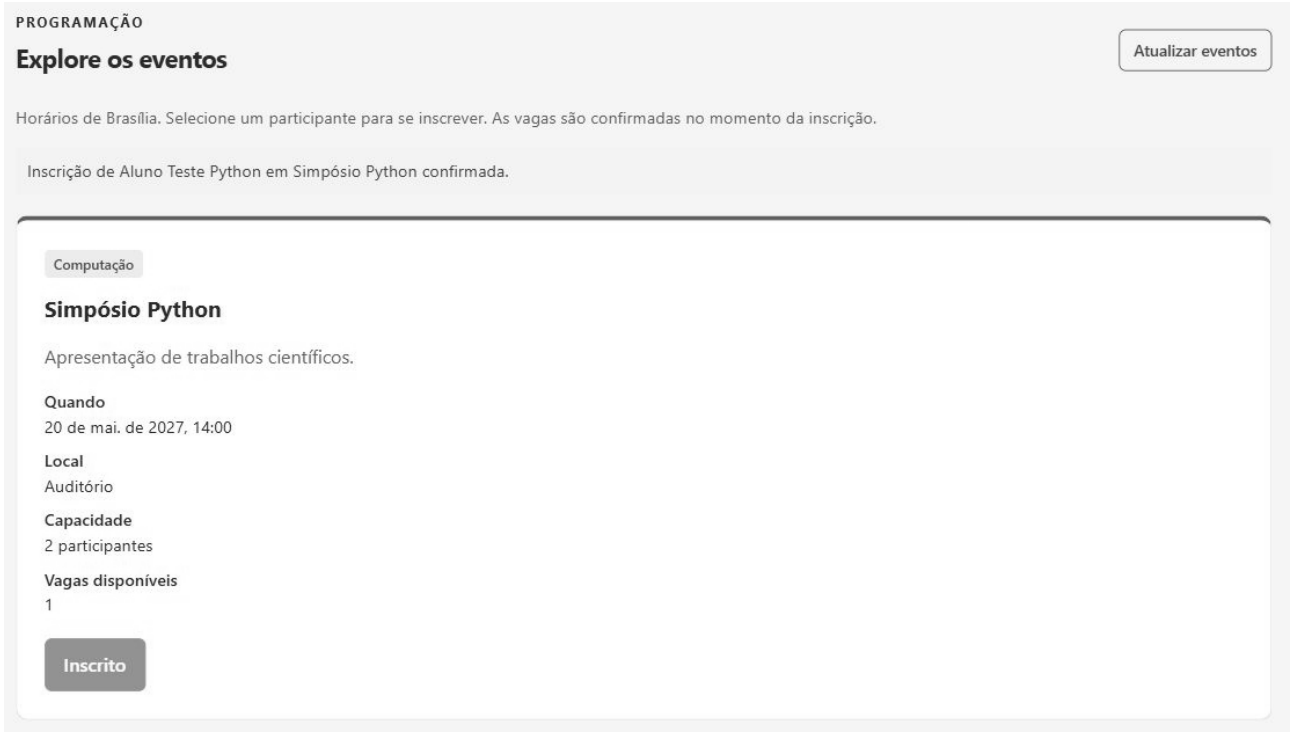


Figura 5 - Consulta das inscrições do participante.



Figura 6 - Evento recomendado antes da inscrição.

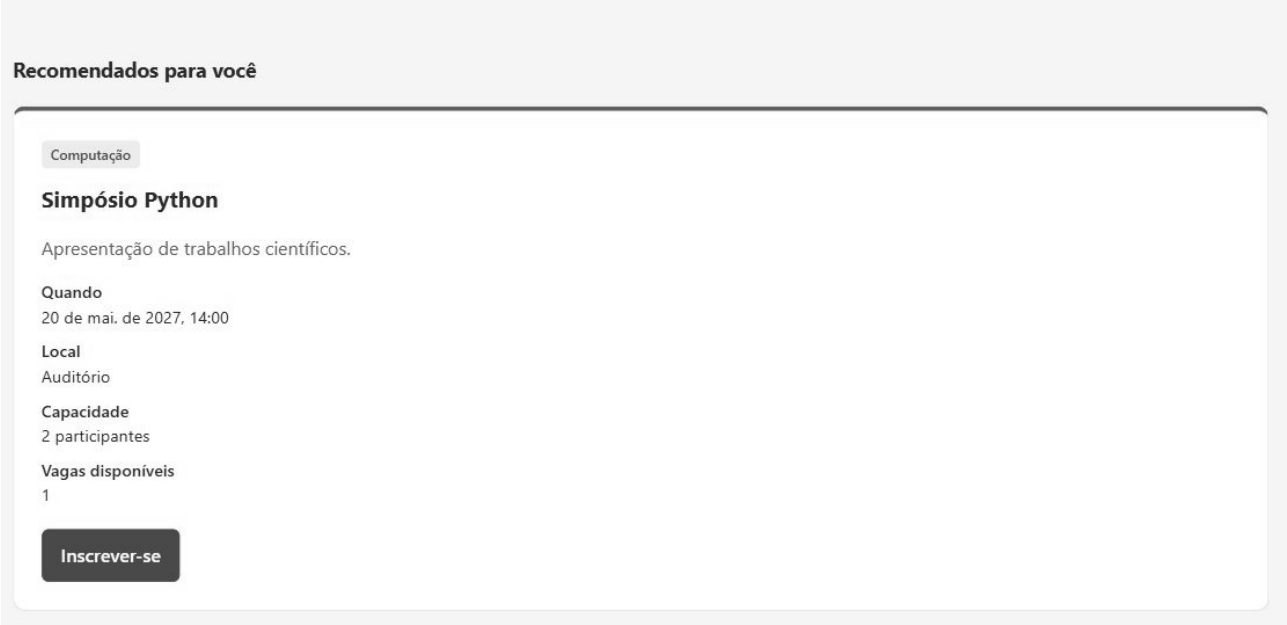
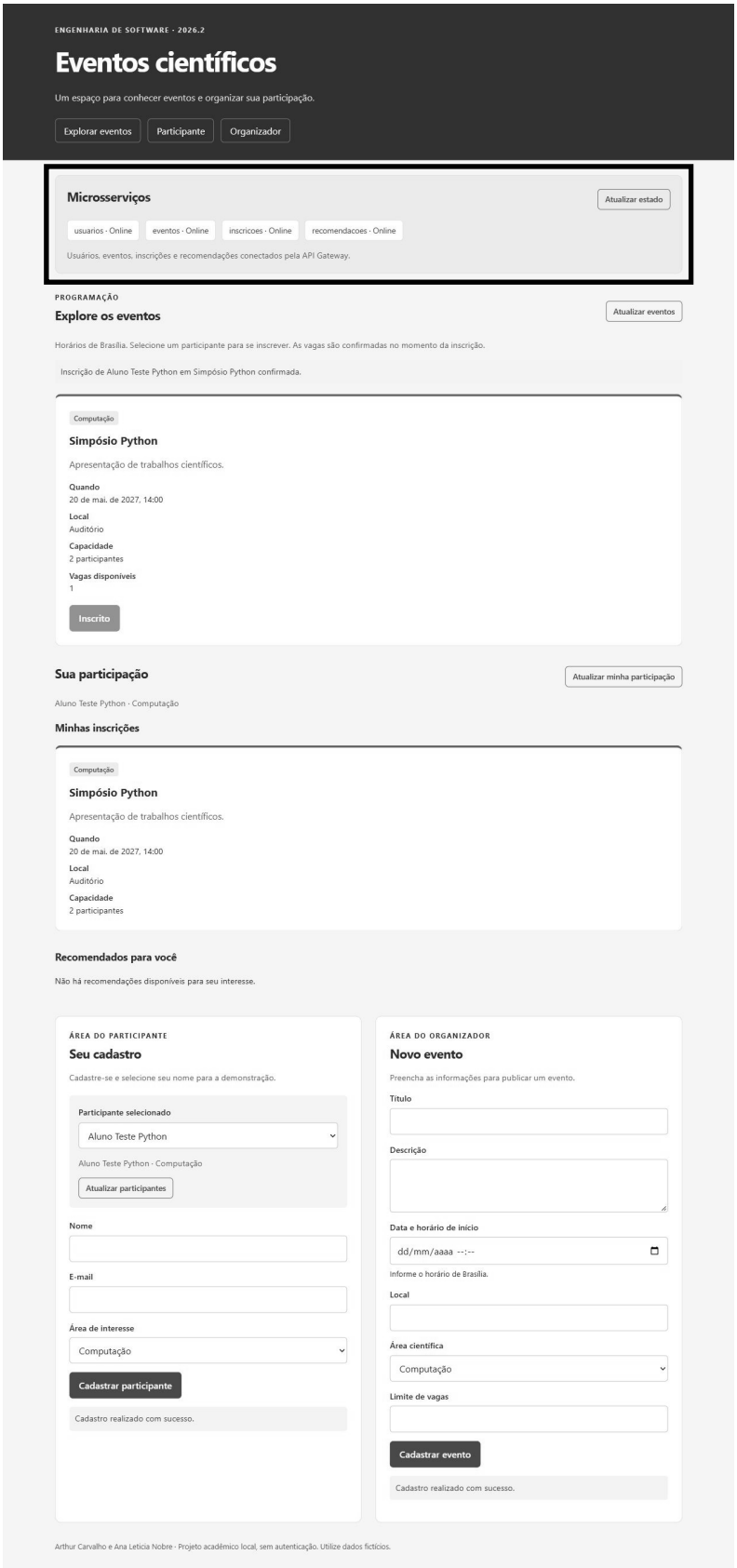


Figura 7 - Página inteira com o painel de microsserviços destacado.



Painel dos serviços

ANEXO B - CÓDIGOS SOLICITADOS

Este anexo reúne o código do Gateway, dos microsserviços, da recomendação e dos componentes reutilizáveis, conforme os itens 6.2, 6.3 e 6.4 das orientações. A descrição das responsabilidades está na seção 7. Os demais arquivos e as instruções de execução estão no repositório.

python/gateway/app.py

```
"""API Gateway com rotas permitidas e entrega da interface existente."""
import asyncio
import re
import httpx
from fastapi import Request
from fastapi.responses import JSONResponse, Response
from fastapi.staticfiles import StaticFiles
from python.common import create_app, error
from python.shared import ROOT, service_urls

app = create_app("gateway")
SERVICES = ("usuarios", "eventos", "inscricoes", "recomendacoes")

@app.get("/api/status")
async def status():
    """Consulta /health dos quatro serviços em paralelo.

    Parâmetros: nenhum. Retorno: lista em dados com nome e estado.
    Falhas individuais são representadas por indisponivel, sem ocultar os demais.
    """
    async def probe(name):
        try:
            async with httpx.AsyncClient(timeout=3) as client:
                response = await client.get(service_urls()[name] + "/health")
            return {"nome": name, "estado": "online" if response.is_success and
    response.json().get("status") == "ok" else "indisponivel"}
        except (httpx.RequestError, ValueError):
            return {"nome": name, "estado": "indisponivel"}
    return {"dados": await asyncio.gather(*(probe(name) for name in SERVICES))}

@app.api_route("/api/{path:path}", methods=["GET", "POST", "PUT", "PATCH", "DELETE"])
async def proxy(path: str, request: Request):
    """Encaminha uma rota pública permitida ao serviço responsável.

    Parâmetros: path é o caminho após /api; request contém método, consulta e corpo.
    Retorno: resposta JSON e código HTTP recebidos do serviço.
    Erros: 404 para rota não permitida; 405 para método inválido;
    503 para conexão indisponível e 504 para tempo excedido.
    """
    # Aceita participantes/eventos com ID opcional e apenas as raízes de
    inscrições/recomendações.
    # Caminhos internos, como /ocupacao, não são expostos.
    match = re.fullmatch(r"(participantes|eventos)(?:/[^/]+)?|(inscricoes|recomendacoes)",
     path)
    if match is None:
        error(404, "ROTA_INEXISTENTE", "Rota não encontrada.")
    name = path.split("/")[0]
    allowed_post = name in ("participantes", "eventos", "inscricoes") and "/" not in path
    if request.method != "GET" and not (request.method == "POST" and allowed_post):
        error(405, "METODO_INVALIDO", "Método não permitido.")
    service = "usuarios" if name == "participantes" else name
    url = service_urls()[service] + "/" + path
    try:
        async with httpx.AsyncClient(timeout=10) as client:
            payload = await request.body() if request.method == "POST" else None
            response = await client.request(request.method, url,
    params=request.query_params, content=payload, headers={"Content-Type":
    "application/json"})
    except httpx.TimeoutException:
        error(504, "TEMPO_EXCEDIDO", "O serviço demorou para responder. Tente novamente.")
```

```
    except httpx.RequestError:
        error(503, "SERVICO_INDISPONIVEL", "Serviço indisponível. Verifique se todos os
    processos estão em execução.")
    return Response(response.content, status_code=response.status_code,
    media_type="application/json")

app.mount("/", StaticFiles(directory=ROOT / "frontend", html=True), name="frontend")
```

python/services/usuarios/app.py

```
"""Cadastro e consulta de participantes; único dono de usuarios.sqlite."""
import re
import sqlite3
from fastapi import Request
from fastapi.responses import JSONResponse
from python.common import area, body, create_app, db, error, ident, required

connection = db("usuarios")
connection.execute("CREATE TABLE IF NOT EXISTS participantes (id INTEGER PRIMARY KEY, nome
     TEXT NOT NULL, email TEXT NOT NULL UNIQUE, area TEXT NOT NULL) STRICT")
app = create_app("usuarios", connection)

@app.get("/participantes")
def list_participants():
    """Lista participantes em ordem de identificador.

    Parâmetros: nenhum. Retorno: lista de cadastros em dados, vazia quando não há
    registros.
    """
    return {"dados": [dict(row) for row in connection.execute("SELECT * FROM participantes
     ORDER BY id")]}

@app.get("/participantes/{participant_id}")
def participant(participant_id: str):
    """Consulta participant_id, um identificador positivo.

    Retorno: cadastro em dados. Erros: 400 para ID inválido e 404 se não existir.
    """
    row = connection.execute("SELECT * FROM participantes WHERE id=?",
    (ident(participant_id),)).fetchone()
    if row is None:
        error(404, "PARTICIPANTE_INEXISTENTE", "Participante não encontrado.")
    return {"dados": dict(row)}

@app.post("/participantes")
async def add_participant(request: Request):
    """Cadastra nome, e-mail e área recebidos no JSON de request.

    Retorno: participante persistido em dados, HTTP 201.
    Erros: 400 para campos inválidos; 409 para e-mail já cadastrado.
    O e-mail é normalizado antes da restrição UNIQUE no banco.
    """
    data = await body(request)
    name = required(data.get("nome"), "Nome", 120)
    email = required(data.get("email"), "E-mail", 254).lower()
    if not re.fullmatch(r"[^\s@]+@[^\s@]+\.[^\s@]+", email):
        error(400, "EMAIL_INVALIDO", "Informe um e-mail válido.")
    interest = area(data.get("area"))
    try:
        cursor = connection.execute("INSERT INTO participantes(nome,email,area)
    VALUES(?,?,?)", (name, email, interest))
    except sqlite3.IntegrityError:
        error(409, "EMAIL_DUPLICADO", "Este e-mail já está cadastrado.")
    row = connection.execute("SELECT * FROM participantes WHERE id=?",
    (cursor.lastrowid,)).fetchone()
    return JSONResponse({"dados": dict(row)}, status_code=201)
```

python/services/eventos/app.py

```
"""Cadastro e consulta de eventos, com vagas obtidas de Inscrições por HTTP."""
from datetime import datetime, timezone
from fastapi import Request
from fastapi.responses import JSONResponse
from python.common import area, body, create_app, db, error, get, ident, iso, required

connection = db("eventos")
connection.execute("CREATE TABLE IF NOT EXISTS eventos (id INTEGER PRIMARY KEY, titulo
    TEXT NOT NULL, descricao TEXT NOT NULL, inicio TEXT NOT NULL, local TEXT NOT NULL,
    area TEXT NOT NULL, capacidade INTEGER NOT NULL CHECK(capacidade>0)) STRICT")
app = create_app("eventos", connection)

@app.get("/eventos")
async def list_events():
    """Lista eventos e calcula vagas pela ocupação de Inscrições.

    Parâmetros: nenhum. Retorno: lista em dados com vagasDisponiveis.
    Erros: 503/504 quando não é possível consultar a ocupação; não inventa vagas.
    """
    rows = [dict(row) for row in connection.execute("SELECT * FROM eventos ORDER BY
    inicio,id")]
    occupancy = {row["eventoId"]: row["total"] for row in await get("inscricoes",
    "/ocupacao")}
    return {"dados": [{**row, "vagasDisponiveis": row["capacidade"] -
    occupancy.get(row["id"], 0)} for row in rows]}

@app.get("/eventos/{event_id}")
def event(event_id: str):
    """Consulta event_id sem consultar a ocupação.

    Retorno: dados básicos do evento. Erros: 400 para ID inválido e 404 se inexistente.
    A consulta básica permite validar inscrições sem uma chamada circular.
    """
    row = connection.execute("SELECT * FROM eventos WHERE id=?",
    (ident(event_id),)).fetchone()
    if row is None:
        error(404, "EVENTO_INEXISTENTE", "Evento não encontrado.")
    return {"dados": dict(row)}

@app.post("/eventos")
async def add_event(request: Request):
    """Cadastra um evento a partir do JSON de request.

    Retorno: evento persistido em dados, HTTP 201; início normalizado em UTC.
    Erros: 400 para campos, área, calendário, fuso, início ou capacidade inválidos.
    """
    data = await body(request)
    title = required(data.get("titulo"), "Título", 160)
    description = required(data.get("descricao"), "Descrição", 2000)
    place = required(data.get("local"), "Local", 200)
    category = area(data.get("area"))
    start = iso(data.get("inicio"))
    if start <= datetime.now(timezone.utc):
        error(400, "DATA_PASSADA", "O evento deve começar no futuro.")
    capacity = data.get("capacidade")
    if isinstance(capacity, bool) or not isinstance(capacity, int) or capacity <= 0:
        error(400, "CAPACIDADE_INVALIDA", "O limite de vagas deve ser um inteiro maior que
     zero.")
    start_text = start.isoformat(timespec="milliseconds").replace("+00:00", "Z")
    cursor = connection.execute("INSERT INTO
    eventos(titulo,descricao,inicio,local,area,capacidade) VALUES(?,?,?,?,?,?)", (title,
    description, start_text, place, category, capacity))
    row = connection.execute("SELECT * FROM eventos WHERE id=?",
    (cursor.lastrowid,)).fetchone()
    return JSONResponse({"dados": dict(row)}, status_code=201)
```

python/services/inscricoes/app.py

```
"""Inscrições com contagem e inserção atômicas no próprio banco."""
import asyncio
import sqlite3
from datetime import datetime, timezone
from fastapi import Request
from fastapi.responses import JSONResponse
from python.common import body, create_app, db, error, get, ident, iso

connection = db("inscricoes")
connection.execute("CREATE TABLE IF NOT EXISTS inscricoes(id INTEGER PRIMARY KEY,
    participanteId INTEGER NOT NULL, eventoId INTEGER NOT NULL, criadaEm TEXT NOT NULL,
    UNIQUE(participanteId,eventoId)) STRICT")
app = create_app("inscricoes", connection)

@app.get("/ocupacao")
def occupancy():
    """Conta inscrições agrupadas por evento no banco proprietário.

    Parâmetros: nenhum. Retorno: lista de eventoId e total, sem chamadas HTTP.
    """
    return {"dados": [dict(row) for row in connection.execute("SELECT eventoId, COUNT(*)
    AS total FROM inscricoes GROUP BY eventoId")]}

@app.get("/inscricoes/ids")
def registration_ids(participanteId: str | None = None):
    """Obtém IDs dos eventos inscritos para participanteId.

    Retorno: lista em dados; erro 400 para identificador inválido.
    Rota interna: não é encaminhada pelo Gateway.
    """
    pid = ident(participanteId)
    return {"dados": [row["eventoId"] for row in connection.execute("SELECT eventoId FROM
    inscricoes WHERE participanteId=?", (pid,))]}

@app.get("/inscricoes")
async def registrations(participanteId: str | None = None):
    """Consulta inscrições do participanteId informado na URL.

    Retorno: registros do participante com os dados de cada evento.
    Erros: 400 para ID inválido, 404 para participante inexistente e
    503/504 se os serviços necessários não responderem.
    """
    pid = ident(participanteId)
    await get("usuarios", f"/participantes/{pid}")
    rows = [dict(row) for row in connection.execute("SELECT * FROM inscricoes WHERE
    participanteId=? ORDER BY id", (pid,))]
    events = await asyncio.gather(*(get("eventos", f"/eventos/{row['eventoId']}") for row
    in rows))
    return {"dados": [{**row, "evento": event} for row, event in zip(rows, events)]}

@app.post("/inscricoes")
async def register(request: Request):
    """Confirma uma inscrição sem ultrapassar a capacidade do evento.

    Parâmetro: request, JSON com participanteId e eventoId.
    Retorno: inscrição criada em dados, HTTP 201.
    Erros: 400 para IDs inválidos; 404 para referências inexistentes;
    409 para duplicidade, início ou lotação; 503/504 para dependências.
    BEGIN IMMEDIATE reserva a escrita antes de contar e inserir.
    COMMIT confirma; qualquer exceção executa ROLLBACK.
    """
    data = await body(request)
    pid, eid = ident(data.get("participanteId")), ident(data.get("eventoId"))
    _, event = await asyncio.gather(get("usuarios", f"/participantes/{pid}"),
    get("eventos", f"/eventos/{eid}"))
    # Não há await nem chamada HTTP durante a transação. A capacidade do evento é
    imutável.
    connection.execute("BEGIN IMMEDIATE")
    try:
```

```
        if iso(event["inicio"]) <= datetime.now(timezone.utc):
            error(409, "EVENTO_INICIADO", "O evento já começou.")
        if connection.execute("SELECT id FROM inscricoes WHERE participanteId=? AND
    eventoId=?", (pid, eid)).fetchone():
            error(409, "INSCRICAO_DUPLICADA", "Você já está inscrito neste evento.")
        total = connection.execute("SELECT COUNT(*) FROM inscricoes WHERE eventoId=?",
    (eid,)).fetchone()[0]
        if total >= event["capacidade"]:
            error(409, "EVENTO_LOTADO", "Não há vagas disponíveis neste evento.")
        timestamp =
    datetime.now(timezone.utc).isoformat(timespec="milliseconds").replace("+00:00", "Z")
        cursor = connection.execute("INSERT INTO
    inscricoes(participanteId,eventoId,criadaEm) VALUES(?,?,?)", (pid, eid, timestamp))
        row = dict(connection.execute("SELECT * FROM inscricoes WHERE id=?",
    (cursor.lastrowid,)).fetchone())
        connection.execute("COMMIT")
    except Exception:
        connection.execute("ROLLBACK")
        raise
    return JSONResponse({"dados": row}, status_code=201)
```

python/services/recomendacoes/app.py

```
"""Sugestões por área, data, vagas e ausência de inscrição anterior."""
import asyncio
from datetime import datetime, timezone
from python.common import create_app, get, ident, iso

app = create_app("recomendacoes")

@app.get("/recomendacoes")
async def recommendations(participanteId: str | None = None):
    """Seleciona sugestões elegíveis para um participante.

    Parâmetro: participanteId, identificador positivo da consulta.
    Retorno: eventos futuros da mesma área, com vagas e sem inscrição prévia,
    ordenados por início e ID. Lista vazia significa ausência de sugestões.
    Erros: 400 para ID inválido, 404 se inexistente, 503/504 se dependências falharem.
    """
    pid = ident(participanteId)
    person = await get("usuarios", f"/participantes/{pid}")
    events, registered = await asyncio.gather(get("eventos", "/eventos"),
    get("inscricoes", "/inscricoes/ids", {"participanteId": pid}))
    excluded = set(registered)
    now = datetime.now(timezone.utc)
    rows = [event for event in events if event["area"] == person["area"] and
    iso(event["inicio"]) > now and event["vagasDisponiveis"] > 0 and event["id"] not in
    excluded]
    return {"dados": sorted(rows, key=lambda event: (event["inicio"], event["id"]))}
```

python/common.py

```
"""Validação, respostas e comunicação compartilhadas pelos serviços."""
import re
import sqlite3
from datetime import datetime, timezone
from urllib.parse import urlencode
import httpx
from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException
from .shared import AREAS, DATA_DIR, service_urls

class Problem(Exception):
    def __init__(self, status, code, message):
        self.status, self.code, self.message = status, code, message

def error(status, code, message):
    raise Problem(status, code, message)

def ident(value):
    if isinstance(value, bool) or not re.fullmatch(r"[1-9][0-9]*", str(value or "")) or
    int(value) > 9007199254740991:
        error(400, "ID_INVALIDO", "O identificador deve ser um inteiro positivo.")
    return int(value)

def required(value, label, limit=200):
    if not isinstance(value, str) or not value.strip() or len(value.strip()) > limit:
        error(400, "CAMPO_INVALIDO", f"{label}: preencha um texto de até {limit}
    caracteres.")
    return value.strip()

def area(value):
    if value not in AREAS:
        error(400, "AREA_INVALIDA", "Selecione uma área válida.")
    return value

def iso(value):
    if not isinstance(value, str) or not
    re.fullmatch(r"\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(?:\.\d{3})?(?:Z|[+-]\d{2}:\d{2})",
    value):
        error(400, "DATA_INVALIDA", "Informe uma data ISO válida com fuso horário.")
    try:
        date = datetime.fromisoformat(value.replace("Z", "+00:00"))
        return date.astimezone(timezone.utc)
    except ValueError:
        error(400, "DATA_INVALIDA", "Informe uma data existente.")

def db(name):
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    connection = sqlite3.connect(DATA_DIR / f"{name}.sqlite", timeout=3,
    check_same_thread=False, isolation_level=None)
    connection.row_factory = sqlite3.Row
    connection.execute("PRAGMA journal_mode=WAL")
    connection.execute("PRAGMA foreign_keys=ON")
    return connection
def create_app(name, connection=None):
    app = FastAPI(title=name, docs_url=None, redoc_url=None, openapi_url=None)
    if connection is not None:
        @app.on_event("shutdown")
        def close_database():
            connection.close()
    @app.exception_handler(Problem)
    async def problem_handler(_request, exc):
        return JSONResponse({"erro": {"codigo": exc.code, "mensagem": exc.message}},
    status_code=exc.status)
    @app.exception_handler(RequestValidationError)
    async def validation_handler(_request, _exc):
        return JSONResponse({"erro": {"codigo": "CAMPO_INVALIDO", "mensagem": "Dados ou
    parâmetros inválidos."}}, status_code=400)
```

```
    @app.exception_handler(StarletteHTTPException)
    async def http_handler(_request, exc):
        code = "ROTA_INEXISTENTE" if exc.status_code == 404 else "METODO_INVALIDO" if
    exc.status_code == 405 else "ERRO_HTTP"
        message = "Rota não encontrada." if exc.status_code == 404 else "Método não
    permitido." if exc.status_code == 405 else "Falha na requisição."
        return JSONResponse({"erro": {"codigo": code, "mensagem": message}},
    status_code=exc.status_code)
    @app.exception_handler(Exception)
    async def unexpected_handler(_request, _exc):
        return JSONResponse({"erro": {"codigo": "ERRO_INTERNO", "mensagem": "Não foi
    possível concluir a operação."}}, status_code=500)
    @app.get("/health")
    def health():
        try:
            if connection is not None:
                connection.execute("SELECT 1").fetchone()
            return {"status": "ok", "servico": name}
        except sqlite3.Error:
            error(503, "BANCO_INDISPONIVEL", "Banco de dados indisponível.")
    return app

async def body(request: Request):
    try:
        if int(request.headers.get("content-length", "0")) > 32768:
            error(413, "CORPO_GRANDE", "O conteúdo enviado é muito grande.")
        result = await request.json()
    except Problem:
        raise
    except Exception:
        error(400, "JSON_INVALIDO", "O corpo deve conter JSON válido.")
    return result if isinstance(result, dict) else {}

async def get(service, path, params=None):
    """Consulta uma dependência por HTTP com limite de três segundos.

    Parâmetros: service identifica o serviço; path é a rota; params é a consulta opcional.
    Retorno: conteúdo do campo dados. Erros de negócio são propagados;
    conexão ou resposta inválida gera 503 e tempo excedido gera 504.
    """
    url = service_urls()[service] + path
    if params:
        url += "?" + urlencode(params)
    try:
        async with httpx.AsyncClient(timeout=3) as client:
            response = await client.get(url)
    except httpx.TimeoutException:
        error(504, "TEMPO_EXCEDIDO", f"O serviço {service} demorou para responder.")
    except httpx.RequestError:
        error(503, "SERVICO_INDISPONIVEL", f"O serviço {service} está indisponível.")
    try:
        result = response.json()
    except ValueError:
        error(503, "SERVICO_INDISPONIVEL", f"O serviço {service} retornou uma resposta
    inválida.")
    if not response.is_success:
        issue = result.get("erro", {})
        error(response.status_code, issue.get("codigo", "DEPENDENCIA_FALHOU"),
    issue.get("mensagem", f"Falha no serviço {service}."))
    return result["dados"]
```

python/shared.py

```
"""Configuração compartilhada dos serviços Python."""

from pathlib import Path
import os

ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = Path(os.environ.get("DATA_DIR", ROOT / "data"))
AREAS = ("Computação", "Engenharias", "Saúde", "Ciências Humanas")

def service_urls():
    """Lê URLs e portas configuradas no ambiente.

    Parâmetros: nenhum. Retorno: dicionário com endereço de cada serviço.
    """
    return {
        name: os.environ.get(f"{name.upper()}_URL",
    f"http://127.0.0.1:{os.environ.get(name.upper() + '_PORT', port)}")
        for name, port in (("usuarios", 3001), ("eventos", 3002), ("inscricoes", 3003),
    ("recomendacoes", 3004))
    }
```

frontend/components/field.js

```
/** Reusable labeled control; textContent avoids interpreting user values as HTML. */
export function field({ name, label, type = 'text', options, maxLength, min, step, hint })
     {
  const wrapper = document.createElement('div');
  wrapper.className = 'field';
  const caption = document.createElement('label');
  caption.htmlFor = name;
  caption.textContent = label;
  const control = document.createElement(options ? 'select' : type === 'textarea' ?
    'textarea' : 'input');
  control.name = name;
  control.id = name;
  control.required = true;
  if (control.tagName === 'INPUT') control.type = type;
  if (maxLength) control.maxLength = maxLength;
  if (min !== undefined) control.min = min;
  if (step !== undefined) control.step = step;
  if (options) {
    const placeholder = new Option('Selecione uma área', '');
    placeholder.disabled = true;
    placeholder.selected = true;
    control.add(placeholder);
    for (const value of options) control.add(new Option(value, value));
  }
  wrapper.append(caption, control);
  if (hint) {
    const help = document.createElement('small');
    help.id = `${name}-help`;
    help.textContent = hint;
    control.setAttribute('aria-describedby', help.id);
    wrapper.append(help);
  }
  return wrapper;
}
```

frontend/components/event-card.js

```
const dateFormat = new Intl.DateTimeFormat('pt-BR', { timeZone: 'America/Sao_Paulo',
    dateStyle: 'medium', timeStyle: 'short' });

/** Cartão reutilizado na programação, nas inscrições e nas recomendações. */
export function eventCard(event, { onRegister, registered = false, participantSelected =
    false, busy = false } = {}) {
  const card = document.createElement('article');
  card.className = 'card';
  const tag = document.createElement('span');
  tag.className = 'tag';
  tag.textContent = event.area;
  const title = document.createElement('h3');
  title.textContent = event.titulo;
  const description = document.createElement('p');
  description.className = 'description';
  description.textContent = event.descricao;
  const details = document.createElement('dl');
  for (const [label, value] of [['Quando', dateFormat.format(new Date(event.inicio))],
    ['Local', event.local], ['Capacidade', `${event.capacidade} participantes`],
    ...(Number.isInteger(event.vagasDisponiveis) ? [['Vagas disponíveis',
    String(event.vagasDisponiveis)]] : [])]) {
    const dt = document.createElement('dt');
    dt.textContent = label;
    const dd = document.createElement('dd');
    dd.textContent = value;
    details.append(dt, dd);
  }
  card.append(tag, title, description, details);
  if (onRegister) {
    const button = document.createElement('button');
    const started = new Date(event.inicio).getTime() <= Date.now();
    button.type = 'button';
    button.textContent = registered ? 'Inscrito' : started ? 'Evento iniciado' :
    event.vagasDisponiveis <= 0 ? 'Sem vagas' : 'Inscrever-se';
    button.disabled = busy || registered || started || event.vagasDisponiveis <= 0 ||
    !participantSelected;
    button.setAttribute('aria-label', `${button.textContent}: ${event.titulo}`);
    button.addEventListener('click', () => onRegister(event));
    card.append(button);
  }
  return card;
}
```

frontend/api.js

```
/** Single reusable HTTP client. All business requests pass through the Gateway. */
export async function api(path, { method = 'GET', body } = {}) {
  let response;
  try {
    response = await fetch(`/api${path}`, {
      method, headers: { 'Content-Type': 'application/json' },
      body: body === undefined ? undefined : JSON.stringify(body),
      signal: AbortSignal.timeout(12000)
    });
  } catch {
    throw new Error('Não foi possível acessar o sistema. Verifique a conexão e os
    serviços.');
  }
  const result = await response.json();
  if (!response.ok) throw new Error(result.erro?.mensagem || 'Não foi possível concluir a
    operação.');
  return result.dados;
}
```